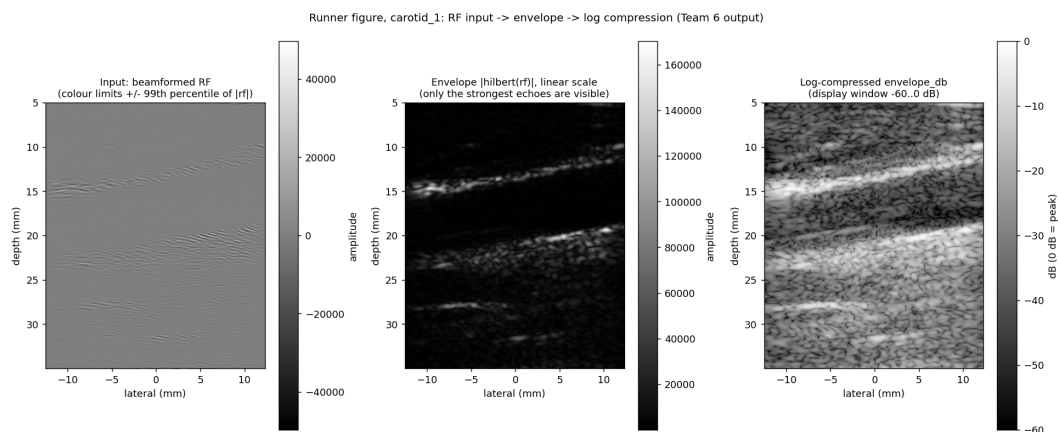


Team 6: Image Reconstruction

From beamformed RF echo lines to the log-compressed envelope (dB)

NITK-USoundSim · 2-D ultrasound simulator · explanation, input, output and run guide

- **Image Reconstruction:** beamformed RF $\rightarrow$ Hilbert envelope $\rightarrow$ normalise to the peak $\rightarrow$ log compression (dB).
- Stage order: **Image Reconstruction (Team 6, this stage)** $\rightarrow$ B-mode Image Formation (Team 7: dynamic range, grayscale image) $\rightarrow$ Post-Image Processing (Team 8: despeckling).
- Main demo on **real in-vivo carotid artery RF data** (steps 1-4 of the team notebook B_mode.ipynb); second demo on simulated RF (image_reconstruction.ipynb: filtering, envelope, scan conversion).
- One command, `python3 run_team6.py`, runs in about 6 s and writes the handoff files `reconstructed_carotid_1.npz` and `reconstructed_carotid_2.npz` for Team 7.
- Measured: carotid_1 echoes span 90.7 dB (strongest / weakest = 34100), carotid_2 86.1 dB.



carotid_1 in this run: RF input, linear envelope, log-compressed envelope (dB).

Contents

1 Overview	3
Files in the folder	3
2 Theory	4
2.1 Beamformed RF data	4
2.2 Analytic signal and Hilbert transform	4
2.3 Envelope	4
2.4 Normalisation to the peak	4
2.5 Log compression (decibels)	5
2.6 Part 2 tools: DC removal and band-pass filtering	5
2.7 Part 2 tools: scan conversion with griddata	5
3 Code explanation	7
3.1 B_mode.ipynb: the part this stage runs	7
3.2 image_reconstruction.ipynb (Part 2)	7
3.3 How run_team6.py uses the team's own code	8
3.4 run_team6.py (runner written for this demo)	9
3.5 Full listing: B_mode.ipynb	10
3.6 Full listing: image_reconstruction.ipynb	12
3.7 Full listing: run_team6.py	17
4 Input	23
4.1 Real in-vivo carotid RF (Part 1)	23
4.2 Simulated RF (Part 2)	24
5 Output	25
5.1 Carotid figures (Part 1)	25
5.2 Handoff files for Team 7	27
5.3 Simulated-RF figures (Part 2)	28
5.4 Console output	28
6 How to run	31
Opening the notebooks yourself	31
Troubleshooting	31
7 Check yourself	32

1 Overview

What Team 6 does and which files are in the folder

The NITK-UsoundSim project simulates a 2-D ultrasound scanner as a chain of stages: transducer, transmit beamforming, wave propagation, tissue interaction and receive beamforming, and then three imaging stages:

Order	Stage	What it does
1	Image Reconstruction (Team 6, this folder)	Beamformed RF -> envelope with the Hilbert transform -> normalise to the peak -> log compression in dB. Hands the log-compressed envelope data to Team 7.
2	B-mode Image Formation (Team 7)	Takes Team 6's dB data, applies the dynamic range, maps it to grayscale brightness 0-1 and forms/saves the B-mode image.
3	Post-Image Processing (Team 8)	Despeckles the finished grayscale image.

The team's carotid notebook `B_mode.ipynb` contains the whole chain from RF to a saved image (its steps 1-9). In the project's stage order its **steps 1-4** (load RF, envelope, normalise, log compression) are this stage's job, and steps 5-9 (dynamic range, brightness, axes, display, save) are Team 7's. `run_team6.py` therefore runs only code cells 1-4 and the first half of code cell 5, without editing the notebook. The second notebook, `image_reconstruction.ipynb`, shows the other reconstruction tools (DC removal, band-pass filtering, envelope and scan conversion) on simulated RF data.

Files in the folder

Path	What it is	Origin
<code>run_team6.py</code>	The one script to run: runs the notebooks' code, saves figures, handoff files and console log	added for this demo
<code>code/B_mode.ipynb</code>	Carotid notebook (9 code cells); this stage runs cells 1-4 and the first half of cell 5	team original, byte-identical copy of <code>teams/T6_B_mode/B_mode_real_carotid_final/B_mode.ipynb</code>
<code>code/image_reconstruction.ipynb</code>	Simulated-RF reconstruction notebook (3 code cells)	team original, unchanged (renamed copy of <code>UsoundIMGRECSample(2).ipynb</code>)
<code>code/carotid_1_rf.npy</code>	Copy of <code>input/carotid_1_rf.npy</code> (identical bytes), placed next to <code>B_mode.ipynb</code> because the notebook loads the bare file name; so the notebook runs directly in Jupyter. <code>run_team6.py</code> does not use it (it reads <code>input/</code>)	copy of the team data, added for this demo
<code>input/carotid_1_rf.npy</code> , <code>input/carotid_2_rf.npy</code>	Real in-vivo beamformed carotid RF, 1218 x 128, float32	team data, copies from <code>B_mode_real_carotid_final/</code>
<code>output/</code>	Everything written by <code>run_team6.py</code> , including the handoff files for Team 7	made by <code>run_team6.py</code>
<code>README.md</code> , <code>requirements.txt</code> , this PDF	Instructions and explanation	added for this demo

2 Theory

The ideas behind the code, in simple words

2.1 Beamformed RF data

After receive beamforming we have a 2-D array $\mathbf{s}(\mathbf{z}, \mathbf{x})$: one column per scan line x (128 here) and one row per depth sample z (1218 here). Each column is an RF signal: a wave that oscillates at about the probe frequency (5 MHz) and whose size tells how strongly the tissue reflects at that depth. The sign of the wave (+ or -) carries no picture information; we want only **how strong** the echo is. The time of a sample gives its depth, because the sound goes down and comes back:

$$z_i = z_0 + \frac{c i}{2f_s}, \quad f_s = 31.25 \text{ MHz}, \quad c = 1540 \text{ m/s}, \quad z_0 = 5 \text{ mm}$$

depth of sample i (factor 2 = round trip). For 1218 samples this gives 5.0 to 35.0 mm.

2.2 Analytic signal and Hilbert transform

The Hilbert transform $\mathcal{H}\{s\}$ shifts every frequency component of the signal by 90 degrees (a cosine becomes a sine). Adding it as an imaginary part gives the **analytic signal**:

$$s_a(z) = s(z) + j\mathcal{H}\{s\}(z)$$

`scipy.signal.hilbert` computes s_a directly with the FFT: it keeps the zero frequency, doubles the positive frequencies, sets the negative frequencies to zero and transforms back. With `axis=0` this is done along depth, separately for every scan line.

2.3 Envelope

$$E(z) = |s_a(z)| = \sqrt{s(z)^2 + \mathcal{H}\{s\}(z)^2}$$

In code: `envelope = np.abs(hilbert(rf, axis=0))`

Simple picture

For a pure wave $A \cos(wt)$ the Hilbert transform is $A \sin(wt)$, so $\sqrt{A^2 \cos^2 + A^2 \sin^2} = A$: the envelope is exactly the amplitude, without the oscillation. For a real echo it is a smooth curve that touches the peaks of the RF wave (figure `carotid_1_01`). The runner checks that the envelope is never below $|rf|$ (result: True for both frames).

2.4 Normalisation to the peak

The envelope values are arbitrary receiver units (`carotid_1`: 4.995831 to 170355.81). Dividing by the largest value of the frame makes the strongest echo exactly 1, so the result no longer depends on the receiver gain or units:

$$E_n(z, x) = \frac{E(z, x)}{\max E}, \quad 0 < E_n \leq 1$$

2.5 Log compression (decibels)

Echo strengths in the body cover a huge range: a vessel wall can reflect thousands of times more than blood or soft tissue. In carotid_1 the strongest envelope value is 34100 times the weakest. On a linear scale only the few strongest echoes are visible and everything else looks black (middle panel of the cover figure). A logarithm compresses this range: equal ratios become equal steps. For amplitudes the decibel uses $20 \log_{10}$ (power is proportional to amplitude squared, and $10 \log_{10}(A^2) = 20 \log_{10}(A)$):

$$L(z, x) = 20 \log_{10}(E_n(z, x) + 10^{-12}) \text{ [dB]}, \quad L \leq 0$$

Envelope relative to the peak	1	1/2	1/10	1/100	1/1000	1/34100 (weakest in carotid_1)
Value in dB	0	-6	-20	-40	-60	-90.65

- **0 dB** is the strongest echo of the frame; every other value is negative: how many dB weaker than the peak.
- Every -20 dB means ten times weaker in amplitude; -6 dB is about half.
- The tiny 10^{-12} only avoids $\log_{10}(0) = -\infty$; it would matter only at -240 dB, far below the weakest real value (-90.65 dB in carotid_1, -86.10 dB in carotid_2).
- In carotid_1 half of all pixels are weaker than -33.75 dB; the histogram figure shows the full spread.

Where this stage stops

Team 6 hands over the log-compressed envelope **without clipping**. Choosing the dynamic range (for example 60 dB), clipping to it and mapping the dB values to gray levels 0-1 is done by the next stage, B-mode Image Formation (Team 7).

2.6 Part 2 tools: DC removal and band-pass filtering

Raw RF can have a constant offset (DC bias) and noise at all frequencies, while the echo energy lies around the probe frequency. `image_reconstruction.ipynb` first subtracts the mean of each scan line (DC removal) and then keeps only 2 to 8 MHz with a **Butterworth band-pass filter** of order 2. `filtfilt` runs the filter forward and then backward, so the phase shifts cancel (zero-phase): the echo does not move in depth. The carotid RF was already band-pass filtered (2-8 MHz) before beamforming, so the carotid notebook does not need this step.

$$|H(f)|^2 = \frac{1}{1 + \left(\frac{f^2 - f_c^2}{fB} \right)^{2n}}$$

shape of a Butterworth band-pass: flat inside the band, falls off outside (n = order, B = bandwidth, f_c = centre)

2.7 Part 2 tools: scan conversion with griddata

The envelope is stored as a table (rows = depth samples, columns = scan lines). A screen needs square pixels on a regular x-z grid. The notebook treats its 64 lines as a fan (sector) from -30 to +30 degrees, so a sample at range r on the line at angle θ lies at

$$x = r \sin \theta, \quad z = r \cos \theta$$

These points are not on a grid, so `scipy.interpolate.griddata` (method 'linear') builds triangles between the known points and reads each pixel of a 200 x 250 grid by linear interpolation inside its triangle. Pixels outside the fan get `fill_value = 0` (black). The carotid data comes from a linear probe

with parallel, vertical scan lines, so for it every column already is a straight vertical line and no fan-shaped scan conversion is needed.

3 Code explanation

The notebooks section by section, how the runner uses them, the runner, then the full listings

3.1 B_mode.ipynb: the part this stage runs

Code cell / section	What it does	Run here?
Cell 1: 1. LOAD REAL RF DATA	Imports numpy, matplotlib, scipy.signal.hilbert. DATA_FILE = "carotid_1_rf.npy"; rf = np.load(DATA_FILE), shape (depth samples, scan lines). Prints shape, min and max.	yes
Cell 2: SAMPLING AND PROBE INFORMATION	fs = 31.25 MHz, f0 = 5 MHz, c = 1540 m/s, pitch = 0.1953 mm, depth_start_mm = 5.0, dynamic_range = 60 (the last value is used only by Team 7's steps).	yes
Cell 3: 2. ENVELOPE DETECTION	analytic_signal = hilbert(rf, axis=0); envelope = np.abs(analytic_signal). Prints shape, min, max.	yes
Cell 4: SHOW ONE SCAN LINE	line = 128 // 2 = 64; depth_axis_mm = 5 + (i / fs) c / 2 * 1000; plots RF and envelope of that line (figure carotid_N_01).	yes
Cell 5, first half: 3. NORMALIZE ENVELOPE	envelope_norm = envelope / np.max(envelope)	yes
Cell 5, first half: 4. LOG COMPRESSION	envelope_db = 20 * np.log10(envelope_norm + 1e-12)	yes
Cell 5, second half: 5. APPLY DYNAMIC RANGE, 6. CONVERT dB TO BRIGHTNESS	clip to [-60, 0] dB and map to 0-1	no (Team 7)
Cells 6-9: axes, display, final image, save	depth/width in mm, 4-panel figure, final B-mode, np.save/plt.imsave	no (Team 7)

How cell 5 is split

Code cell 5 holds steps 3-6. The runner reads its text and finds the line containing # 5. APPLY DYNAMIC RANGE (line 16 of the cell). That heading is a three-line comment block ('=' line, title, '=' line), so the '=' line just above it (line 15) is the start of the second half. The first half, lines 1-14, contains exactly the sections 3. NORMALIZE ENVELOPE and 4. LOG COMPRESSION and is run; the second half (dynamic range and brightness) is not run. The notebook file itself is never edited.

Note: section 5 of the notebook overwrites envelope_db with its clipped version. Because the runner stops before it, envelope_db in the handoff file is the unclipped log-compressed envelope.

3.2 image_reconstruction.ipynb (Part 2)

The notebook has 3 code cells. Cell 0 holds only a comment with a reference link (removed in the listing). Cell 2 is empty. All the work is in cell 1:

Section	What it does
STEP 1 parameters	fs = 40 MHz, fc = 5 MHz, c = 1540 m/s, 64 lines, 80 us -> 3201 samples; random seed 42.
Targets and echoes	Point targets at 15 mm (line 32, strength 1.0), 30 mm (line 20, 0.8), 45 mm (line 45, 0.6). Each gives a Gaussian 5 MHz burst at $t = 2z/c$, spread over neighbouring lines with weight $\exp(-d^2/(2*4^2))$.
DC bias + noise	Adds an offset of 0.08 and Gaussian noise (std 0.05) -> raw_rf_input.
STEP 2.1 DC removal	Subtracts the mean of every scan line.

Section	What it does
STEP 2.2 band-pass	butter(2, [2, 8] MHz / Nyquist) and filtfilt along depth -> preprocessed_rf.
STEP 3 envelope	hilbert() along depth, then abs() -> envelope.
STEP 4 depth	depth_mm = c t / 2 * 1000 (0 to 61.6 mm).
STEP 5 lateral	lateral_positions_mm (0.5 mm pitch) is computed but not used later; E_xz = envelope.
STEP 6 scan conversion	Angles -30..+30 deg; $x = r \sin(\theta)$, $z = r \cos(\theta)$; griddata (linear) onto a 200 x 250 grid, 0 outside.
Visualisation, final output	RF + envelope of line 32 and the reconstructed image (plt.show); prints shape and ranges.

3.3 How run_team6.py uses the team's own code

The algorithm is the team's code

Everything that turns RF into the reconstructed data comes from the team's own notebooks: the Hilbert envelope, the normalisation to the peak and the log compression from code/B_mode.ipynb, and the DC removal, band-pass filter, envelope and scan conversion from code/image_reconstruction.ipynb.

run_team6.py reads the code cells of these notebooks and runs them cell by cell, **unchanged**.

The notebook files are never edited. run_team6.py itself is only a demo wrapper: it runs the cells, saves the results and adds checks and extra figures.

Where run_team6.py uses your code

run_team6.py line(s)	Notebook and code cell used	What it does there
77-79	both notebooks, all code cells	code_cells() opens the .ipynb file as JSON and returns the text of every code cell exactly as written.
93	B_mode.ipynb, all 9 code cells	Reads the carotid notebook's code cells into bm_cells.
101-105	B_mode.ipynb, cell 5	Splits cell 5 at the line # 5. APPLY DYNAMIC RANGE (the '=' line above it starts the second half). First half = 3. NORMALIZE ENVELOPE and 4. LOG COMPRESSION (run); second half = Team 7's part (not run).
117	B_mode.ipynb, cells 1, 2, 3, 4 + first half of cell 5	Chooses the cells this stage runs: load RF, probe values, Hilbert envelope, scan-line plot, normalise, log compression.
118-121	B_mode.ipynb, cell 1	For carotid_2 only: replaces the line DATA_FILE = "carotid_1_rf.npy" by DATA_FILE = "carotid_2_rf.npy" in memory. The notebook file is not changed.
126-131, 135	B_mode.ipynb, cell 4	Replaces plt.show by a function that saves the figure drawn by cell 4 as carotid_N_01_notebook_rf_and_envelope.png.
136	B_mode.ipynb, cell 1	Sets the working folder to input/, so that np.load(DATA_FILE) in cell 1 finds input/carotid_N_rf.npy.
139-141	B_mode.ipynb, cells 1-4 + first half of cell 5	Runs the chosen cells one after another with exec() in one shared namespace, like running them in Jupyter.
111, 147, 165, 177	B_mode.ipynb, cells 1-5 (variables)	Takes the notebook's own results: rf (cell 1), fs, f0, c, pitch, depth_start_mm (cell 2), envelope (cell 3), depth_axis_mm (cell 4), envelope_norm and envelope_db (cell 5).
220	image_reconstruction.ipynb, all 3 code cells	Reads the second notebook's code cells.

run_team6.py line(s)	Notebook and code cell used	What it does there
224-227, 232	image_reconstruction.ipynb, cell 1	Replaces plt.show so the notebook's own figure is saved as sim_A1_notebook_rf_envelope_and_scan_conversion.png.
235-237	image_reconstruction.ipynb, cells 0, 1, 2	Runs all cells unchanged: simulated RF, DC removal, band-pass filter, Hilbert envelope, scan conversion.

What run_team6.py adds on top (written for this demo, not the team's code)

run_team6.py line(s)	What it adds	Result
46-62, 303-305	Copies everything printed to the screen into a log.	output/console_log.txt
308-325	Draws the console text as a terminal-style picture.	output/console_screenshot.png
106-108	Prints where cell 5 is split.	console text
148-160	Checks on the notebook's variables: shape, dtype, min, max; envelope $\geq rf $ everywhere (line 152); strongest / weakest echo in dB; envelope_db min, max, median; % of pixels below -40, -50, -60 dB.	console text
162-174	Saves the handoff file for Team 7 with np.savez (rf, envelope, envelope_norm, envelope_db + the cell-2 values) and reads it back to print what is inside.	output/reconstructed_carotid_N.npz
176-197	Runner figure: RF input, linear envelope, log-compressed envelope (dB). The lateral axis uses the same formula as notebook cell 6, typed into the runner (cell 6 itself is not run).	carotid_N_02_runner_rf_envelope_db.png
199-214	Runner figure: envelope_db over its full range and a histogram of its values.	carotid_N_03_runner_db_full_range_and_histogram.png
242-256	Part 2 checks: RF size, filter settings, mean (DC) before and after filtering, size and zero-filled % of the scan-converted image. Target-peak check (lines 249-253): for each of the 3 point targets, finds the depth of the envelope peak on its scan line.	console text
258-272	Runner figure: raw simulated RF of line 32 before/after filtering and the whole RF matrix.	sim_A2_runner_raw_rf_input.png
274-288	Runner figure: envelope and scan-converted image converted to dB by the runner (the notebook itself stops at the linear envelope).	sim_A3_runner_envelope_db_and_scan_converted.png
291-300	Summary: handoff files, number of files written, total run time.	console text

To see it yourself: open run_team6.py at the lines above; open code/B_mode.ipynb to see the same cells.

3.4 run_team6.py (runner written for this demo)

- Sends all printed text to the screen and to output/console_log.txt; uses matplotlib's Agg backend and replaces plt.show by a function that saves the current figure into output/.
- **Part 1:** reads B_mode.ipynb as JSON, splits code cell 5 as explained above and, for carotid_1 and carotid_2, runs cells 1-4 and the first half of cell 5 with exec in a fresh namespace, with the working directory set to input/ so that the bare file name is found. For carotid_2 only the line DATA_FILE = "carotid_1_rf.npy" is replaced in memory.
- Prints checks on the notebook's variables (shape, dtype, range, dB statistics), saves the handoff file with np.savez and draws two runner figures per frame.

- **Part 2:** runs all code cells of image_reconstruction.ipynb unchanged, prints checks (filter, DC, peak depths) and draws two runner figures.
- Finally it writes the console log and a screenshot-style PNG of it (its own small function; the folder imports nothing from outside).

3.5 Full listing: B_mode.ipynb

All 9 code cells are listed. Cells 1-4 and cell 5 lines 1-14 are **run by this stage**; the rest is shown for completeness and belongs to Team 7.

B_mode.ipynb code cell 1 [RUN by this stage]

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.signal import hilbert
4
5
6 # =====
7 # 1. LOAD REAL RF DATA
8 # =====
9
10 DATA_FILE = "carotid_1_rf.npy"      # change to "carotid_2_rf.npy" for the second image
11
12 rf = np.load(DATA_FILE)              # shape: (depth samples, scan lines)
13
14 print("RF data shape:", rf.shape)
15 print("Depth samples:", rf.shape[0])
16 print("Scan lines   :", rf.shape[1])
17 print("Minimum:", rf.min())
18 print("Maximum:", rf.max())
```

B_mode.ipynb code cell 2 [RUN by this stage]

```
1 # =====
2 # SAMPLING AND PROBE INFORMATION (real probe)
3 # =====
4
5 fs = 31.25e6          # Sampling frequency = 31.25 MHz
6 f0 = 5e6              # Centre frequency of probe = 5 MHz
7 c = 1540              # Speed of sound in tissue = 1540 m/s
8 pitch = 0.1953e-3     # Distance between scan lines = 0.1953 mm
9 depth_start_mm = 5.0  # First sample is at 5 mm depth
10
11 dynamic_range = 60    # 60 dB
```

B_mode.ipynb code cell 3 [RUN by this stage]

```
1 # =====
2 # 2. ENVELOPE DETECTION (HILBERT TRANSFORM)
3 # =====
4 # Hilbert transform is applied along depth (axis=0) for every scan line.
5 # abs() of the analytic signal gives the envelope (echo strength).
6
7 analytic_signal = hilbert(rf, axis=0)
8 envelope = np.abs(analytic_signal)
9
10 print("Envelope shape:", envelope.shape)
11 print("Minimum:", envelope.min())
12 print("Maximum:", envelope.max())
```

B_mode.ipynb code cell 4 [RUN by this stage]

```

1 # =====
2 # SHOW ONE SCAN LINE: RF SIGNAL AND ITS ENVELOPE
3 # =====
4
5 line = rf.shape[1] // 2 # middle scan line
6 depth_axis_mm = depth_start_mm + (np.arange(rf.shape[0]) / fs) * c / 2 * 1000
7
8 plt.figure(figsize=(12, 4))
9 plt.plot(depth_axis_mm, rf[:, line], linewidth=0.7, label="RF signal")
10 plt.plot(depth_axis_mm, envelope[:, line], linewidth=1.5, label="Envelope (Hilbert)")
11 plt.xlabel("Depth (mm)")
12 plt.ylabel("Amplitude")
13 plt.title(f"Scan line {line}: RF signal and envelope")
14 plt.legend()
15 plt.show()

```

B_mode.ipynb code cell 5, lines 1-14 [RUN by this stage]

```

1 # =====
2 # 3. NORMALIZE ENVELOPE
3 # =====
4
5 envelope_norm = envelope / np.max(envelope)
6
7
8 # =====
9 # 4. LOG COMPRESSION (CONVERT TO dB)
10 # =====
11
12 envelope_db = 20 * np.log10(envelope_norm + 1e-12)

```

B_mode.ipynb code cell 5, lines 15-29 [NOT run here: Team 7]

```

1 # =====
2 # 5. APPLY DYNAMIC RANGE
3 # =====
4
5 envelope_db = np.clip(envelope_db, -dynamic_range, 0)
6
7
8 # =====
9 # 6. CONVERT dB TO BRIGHTNESS (0 to 1)
10 # =====
11
12 bmode = (envelope_db + dynamic_range) / dynamic_range
13
14 print("B-mode shape:", bmode.shape)
15 print("Brightness range:", bmode.min(), "to", bmode.max())

```

B_mode.ipynb code cell 6 [NOT run here: Team 7]

```

1 # =====
2 # 7. CALCULATE DEPTH AND WIDTH (mm)
3 # =====
4
5 num_samples, num_scanlines = rf.shape
6
7 sample_indices = np.arange(num_samples)
8 time = sample_indices / fs # time of each sample (s)
9 depth = (c * time) / 2 # divide by 2: sound goes and comes back
10 depth_mm = depth_start_mm + depth * 1000
11
12 width_mm = (np.arange(num_scanlines) - num_scanlines / 2) * pitch * 1000
13
14 print("Depth: %.1f mm to %.1f mm" % (depth_mm[0], depth_mm[-1]))
15 print("Width: %.1f mm to %.1f mm" % (width_mm[0], width_mm[-1]))
16
17 extent = [width_mm[0], width_mm[-1], depth_mm[-1], depth_mm[0]]

```

B_mode.ipynb code cell 7 [NOT run here: Team 7]

```

1 # =====
2 # 8. DISPLAY ALL STEPS
3 # =====
4
5 fig, ax = plt.subplots(1, 4, figsize=(20, 6))
6
7 im = ax[0].imshow(rf, cmap="gray", extent=extent)
8 ax[0].set_title("1. Beamformed RF data")
9 plt.colorbar(im, ax=ax[0])
10
11 im = ax[1].imshow(envelope, cmap="gray", extent=extent)
12 ax[1].set_title("2. Envelope (after Hilbert)")
13 plt.colorbar(im, ax=ax[1])
14
15 im = ax[2].imshow(envelope_db, cmap="gray", vmin=-dynamic_range, vmax=0, extent=extent)
16 ax[2].set_title("3. Log-compressed (dB)")
17 plt.colorbar(im, ax=ax[2], label="dB")
18
19 im = ax[3].imshow(bmode, cmap="gray", vmin=0, vmax=1, extent=extent)
20 ax[3].set_title("4. Final B-mode")
21 plt.colorbar(im, ax=ax[3], label="Brightness")
22
23 for a in ax:
24     a.set_xlabel("Lateral (mm)")
25     a.set_ylabel("Depth (mm)")
26
27 plt.tight_layout()
28 plt.show()

```

B_mode.ipynb code cell 8 [NOT run here: Team 7]

```

1 # =====
2 # FINAL B-MODE IMAGE
3 # =====
4
5 plt.figure(figsize=(7, 8))
6
7 plt.imshow(bmode, cmap="gray", vmin=0, vmax=1, extent=extent)
8
9 plt.xlabel("Lateral (mm)")
10 plt.ylabel("Depth (mm)")
11 plt.title("Final B-mode Ultrasound Image\nReal in-vivo carotid artery (60 dB)")
12 plt.colorbar(label="Brightness")
13
14 plt.show()

```

B_mode.ipynb code cell 9 [NOT run here: Team 7]

```

1 # =====
2 # 9. SAVE OUTPUT FOR IMAGE RECONSTRUCTION / POST-PROCESSING TEAM
3 # =====
4
5 name = DATA_FILE.replace("_rf.npy", "")
6
7 np.save(f"bmode_{name}.npy", bmode.astype(np.float32)) # float values 0 to 1
8 plt.imsave(f"bmode_{name}.png", bmode, cmap="gray", vmin=0, vmax=1) # grayscale image
9 np.savez(f"bmode_{name}_axes.npz", depth_mm=depth_mm, width_mm=width_mm)
10
11 print("Saved:", f"bmode_{name}.npy", "|", f"bmode_{name}.png", "|", f"bmode_{name}_axes.npz")
12 print("Image size:", bmode.shape, "(depth samples x scan lines)")

```

3.6 Full listing: image_reconstruction.ipynb

image_reconstruction.ipynb code cell 0

```
1 # (reference link removed)
```

image_reconstruction.ipynb code cell 1

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.signal import butter, filtfilt, hilbert
4 from scipy.interpolate import griddata
5
6 # =====
7 # STEP 1: Beamformed RF Data Input
8 # =====
9 # We receive beamformed RF data from the beamforming team.
10 # Here, we simulate a 5 MHz center-frequency transducer
11 # with 64 ultrasound scan lines.
12
13 fs = 40e6          # Sampling frequency = 40 MHz
14 fc = 5e6           # Transducer center frequency = 5 MHz
15 c = 1540           # Speed of sound in tissue = 1540 m/s
16 num_lines = 64     # Number of scan lines
17 t_max = 80e-6      # Maximum acquisition time = 80 microseconds
18
19
20 # Time samples
21 t = np.arange(0, t_max, 1 / fs)
22 num_samples = len(t)
23
24 # RF data matrix: Samples × Scanlines
25 np.random.seed(42)
26 rf_matrix = np.zeros((num_samples, num_lines))
27
28
29 # =====
30 # Define Simulated Point Targets
31 # =====
32 # Format:
33 # (depth in meters, scan-line index, reflectivity)
34
35 targets = [
36     (0.015, 32, 1.0),    # Target at 15 mm
37     (0.030, 20, 0.8),    # Target at 30 mm
38     (0.045, 45, 0.6)     # Target at 45 mm
39 ]
40
41 # =====
42 # Generate Simulated RF Echoes
43 # =====
44
45 for depth, line_idx, strength in targets:
46     # Round-trip echo arrival time
47     t_echo = (2 * depth) / c
48
49     # Pulse duration
50     pulse_width = 1.0e-6
51
52     for l in range(num_lines):
53         # Distance from target's main scan line
54         lateral_distance = abs(l - line_idx)
55
56         # Simulate lateral spreading of the echo
57         lateral_attenuation = np.exp(
58             -(lateral_distance ** 2) / (2 * 4.0 ** 2)
59         )
60
61         if lateral_attenuation > 0.01:
62             # Gaussian-modulated RF carrier
63             pulse = (
64                 strength
65                 * lateral_attenuation
66                 * np.exp(
67                     -((t - t_echo) ** 2)
```

```

71         / (2 * pulse_width ** 2)
72     )
73     * np.cos(
74         2 * np.pi * fc * (t - t_echo)
75     )
76 )
77
78     rf_matrix[:, l] += pulse
79
80 # =====
81 # Add DC Bias and Random Noise
82 # =====
83
84 dc_bias = 0.08
85
86 noise = np.random.normal(
87     0,
88     0.05,
89     rf_matrix.shape
90 )
91
92 raw_rf_input = rf_matrix + dc_bias + noise
93
94 # =====
95 # STEP 2: RF Data Preprocessing
96 # =====
97
98 # -----
99 # 2.1 DC Offset Removal
100 # -----
101
102 rf_no_dc = (
103     raw_rf_input
104     - np.mean(raw_rf_input, axis=0)
105 )
106
107 # -----
108 # 2.2 Bandpass Filtering
109 # -----
110
111 low_cut = 2.0e6
112 high_cut = 8.0e6
113
114 nyquist = 0.5 * fs
115
116 b, a = butter(
117     2,
118     [
119         low_cut / nyquist,
120         high_cut / nyquist
121     ],
122     btype='bandpass'
123 )
124
125 # Zero-phase filtering
126 preprocessed_rf = filtfilt(
127     b,
128     a,
129     rf_no_dc,
130     axis=0
131 )
132
133 # =====
134 # STEP 3: Envelope Detection
135 # =====
136 # Use Hilbert Transform to obtain the
137 # amplitude envelope of each RF scan line.
138
139 analytic_signal = hilbert(
140     preprocessed_rf,
141     axis=0
142 )
143
144 # Envelope = magnitude of analytic signal
145 envelope = np.abs(analytic_signal)
146
147 # =====

```

```

148 # STEP 4: Time-to-Depth Conversion
149 # =====
150 # z = c*t/2
151 #
152 # Factor of 2 is because ultrasound travels
153 # from the transducer to the target and back.
154
155 depth_mm = (
156     (c * t) / 2
157 ) * 1000
158
159 # =====
160 # STEP 5: Spatial Reconstruction
161 # =====
162 # Organize the envelope data according to
163 # lateral scan-line positions.
164
165 lateral_pitch_mm = 0.5
166
167 lateral_positions_mm = (
168     np.arange(num_lines)
169     * lateral_pitch_mm
170 )
171
172 # Reconstructed envelope in X-Z coordinates
173 E_xz = envelope
174
175 # =====
176 # STEP 6: Scan Conversion
177 # =====
178 # Convert the scan-line representation into
179 # a regular Cartesian spatial grid.
180
181 theta = np.linspace(
182     -30,
183     30,
184     num_lines
185 ) * np.pi / 180
186
187 theta_grid, r_grid = np.meshgrid(
188     theta,
189     depth_mm
190 )
191
192 # Polar → Cartesian coordinates
193
194 x_polar = (
195     r_grid
196     * np.sin(theta_grid)
197 )
198
199 z_polar = (
200     r_grid
201     * np.cos(theta_grid)
202 )
203
204
205 # -----
206 # Define Cartesian Grid
207 # -----
208
209 x_cart = np.linspace(
210     np.min(x_polar),
211     np.max(x_polar),
212     200
213 )
214
215 z_cart = np.linspace(
216     0,
217     np.max(z_polar),
218     250
219 )
220
221 X_grid, Z_grid = np.meshgrid(
222     x_cart,
223     z_cart
224 )

```

```

225 )
226
227 # -----
228 # Interpolation
229 # -----
230
231 points = np.vstack(
232     (
233         x_polar.flatten(),
234         z_polar.flatten()
235     )
236 ).T
237
238 values = E_xz.flatten()
239
240 # Final reconstructed envelope
241 reconstructed_envelope = griddata(
242     points,
243     values,
244     (X_grid, Z_grid),
245     method='linear',
246     fill_value=0.0
247 )
248
249 # =====
250 # VISUALIZATION
251 # =====
252
253 fig, axs = plt.subplots(
254     1,
255     2,
256     figsize=(12, 6)
257 )
258
259
260 # -----
261 # Plot 1: RF Signal + Envelope
262 # -----
263
264 axs[0].plot(
265     depth_mm,
266     preprocessed_rf[:, 32],
267     label="Preprocessed RF Signal",
268     alpha=0.5
269 )
270
271 axs[0].plot(
272     depth_mm,
273     envelope[:, 32],
274     label="Hilbert Envelope",
275     linewidth=1.8
276 )
277
278 axs[0].set_title(
279     "RF Signal and Envelope - Scan Line 32"
280 )
281
282 axs[0].set_xlabel(
283     "Depth (mm)"
284 )
285
286 axs[0].set_ylabel(
287     "Amplitude"
288 )
289
290 axs[0].grid(True)
291 axs[0].legend()
292
293 # -----
294 # Plot 2: Reconstructed Envelope
295 # -----
296
297 axs[1].imshow(
298     reconstructed_envelope,
299     extent=[
300         x_cart[0],
301         x_cart[-1],

```



```

302         z_cart[-1],
303         z_cart[0]
304     ],
305     aspect='auto'
306 )
307
308 axs[1].set_title(
309     "Reconstructed Envelope Data"
310 )
311
312 axs[1].set_xlabel(
313     "Lateral Position (mm)"
314 )
315
316 axs[1].set_ylabel(
317     "Depth (mm)"
318 )
319
320
321 plt.tight_layout()
322 plt.show()
323
324
325 # =====
326 # FINAL OUTPUT
327 # =====
328
329 print(
330     "Reconstructed envelope shape:",
331     reconstructed_envelope.shape
332 )
333
334 print(
335     "Depth range:",
336     depth_mm[0],
337     "to",
338     depth_mm[-1],
339     "mm"
340 )
341
342 print(
343     "Lateral range:",
344     x_cart[0],
345     "to",
346     x_cart[-1],
347     "mm"
348 )
349

```

image_reconstruction.ipynb code cell 2: empty.

In cell 0 the original comment line is a web reference link; in this listing it is replaced by '# (reference link removed)'. The file in code/ is unchanged.

3.7 Full listing: run_team6.py

run_team6.py

```

1  """
2  run_team6.py - Team 6 demo: Image Reconstruction (NITK-UsoundSim).
3
4  Stage order in the project:
5      Image Reconstruction (Team 6, THIS folder): Hilbert envelope -> normalisation -> log compressi
    ↳ on (dB)
6      -> B-mode Image Formation (Team 7): dynamic range, grayscale brightness 0-1, final B-mode imag
    ↳ e
7      -> Post-Image Processing (Team 8): despeckling of the finished image
8
9  One command runs the stage and saves every result into output/:
10
11      python3 run_team6.py          (macOS / Linux)
12      python  run_team6.py          (Windows)
13
14  Part 1 (main demo, real in-vivo carotid RF data, code/B_mode.ipynb):
15      runs code cells 1, 2, 3, 4 of the notebook in order, then ONLY the first half of code cell 5

```

```

16     (sections "3. NORMALIZE ENVELOPE" and "4. LOG COMPRESSION"). The second half of cell 5 (dynam
    ↳ c range and
17     brightness) and cells 6-9 belong to Team 7 and are not run here. This is done for carotid_1 an
    ↳ d carotid_2.
18     The result is saved as the handoff for Team 7: output/reconstructed_carotid_1.npz, output/reco
    ↳ nstructed_carotid_2.npz
19
20 Part 2 (the team's second notebook, code/image_reconstruction.ipynb):
21     simulated 5 MHz RF -> DC removal -> band-pass filter -> Hilbert envelope -> scan conversion.
22
23 The notebooks in code/ are NOT changed; this script only reads them and runs their code.
24 """
25 import io
26 import json
27 import os
28 import sys
29 import time
30 from pathlib import Path
31
32 import matplotlib
33
34 matplotlib.use("Agg") # save figures to files, do not open windows
35 import matplotlib.pyplot as plt # noqa: E402
36 import numpy as np # noqa: E402
37
38 HERE = Path(__file__).resolve().parent
39 CODE, INPUT, OUTPUT = HERE / "code", HERE / "input", HERE / "output"
40 OUTPUT.mkdir(exist_ok=True)
41
42
43 # -----
44 # Console helper: everything printed goes to the screen AND to output/console_log.txt
45 # -----
46 class Tee(io.TextIOBase):
47     def __init__(self, *streams):
48         self.streams = streams
49
50     def write(self, s):
51         for st in self.streams:
52             st.write(s)
53         return len(s)
54
55     def flush(self):
56         for st in self.streams:
57             st.flush()
58
59
60 LOG = io.StringIO()
61 REAL_STDOUT = sys.stdout
62 sys.stdout = Tee(REAL_STDOUT, LOG)
63 SAVED = [] # list of (file name, meaning)
64
65
66 def save(fig, name, meaning, dpi=130):
67     fig.savefig(OUTPUT / name, dpi=dpi, bbox_inches="tight")
68     plt.close(fig)
69     SAVED.append((name, meaning))
70     print(f"  saved output/{name}")
71
72
73 def banner(text):
74     print("\n" + "=" * 78 + f"\n{text}\n" + "=" * 78)
75
76
77 def code_cells(nb_path):
78     nb = json.loads(nb_path.read_text(encoding="utf-8"))
79     return ["".join(c["source"]) for c in nb["cells"] if c["cell_type"] == "code"]
80
81
82 t_start = time.time()
83 banner("TEAM 6: IMAGE RECONSTRUCTION (NITK-UsoundSim)")
84 print(f"Python {sys.version.split()[0]}, numpy {np.__version__}, matplotlib {matplotlib.__version__}
    ↳ ")
85 print("Stage order: Image Reconstruction (Team 6: Hilbert envelope + log compression)")
86 print("      -> B-mode Image Formation (Team 7: dynamic range + grayscale image)")
87 print("      -> Post-Image Processing (Team 8: despeckling)")
88

```

```

89 # =====
90 # PART 1: real carotid RF data, code/B_mode.ipynb, steps 1-4 only
91 # =====
92 banner("PART 1: real in-vivo carotid RF (code/B_mode.ipynb, steps 1-4)")
93 bm_cells = code_cells(CODE / "B_mode.ipynb")
94 print(f"B_mode.ipynb has {len(bm_cells)} code cells. This stage runs cells 1, 2, 3, 4 and the first half of cell 5.")
95
96 # Split code cell 5 at the "# 5. APPLY DYNAMIC RANGE" heading. The heading is a 3-line comment block
97 # # =====
98 # # 5. APPLY DYNAMIC RANGE
99 # # =====
100 # so the '=' line just above the marker already belongs to the second half (Team 7's part).
101 cell5_lines = bm_cells[4].split("\n")
102 k = next(i for i, ln in enumerate(cell5_lines) if "# 5. APPLY DYNAMIC RANGE" in ln)
103 assert cell5_lines[k - 1].strip().startswith("# ====="), "unexpected layout of code cell 5"
104 cell5_part1 = "\n".join(cell5_lines[:k - 1]).rstrip() + "\n" # sections 3 (normalise) and 4 (log compression)
105 cell5_part2 = "\n".join(cell5_lines[k - 1:]) # sections 5 and 6: Team 7, NOT run here
106 print(f"Code cell 5 split at line {k} (the '=' line above '5. APPLY DYNAMIC RANGE'):")
107 print(f"first half = lines 1-{k - 1} (3. NORMALIZE ENVELOPE, 4. LOG COMPRESSION) -> run here")
108 print(f"second half = lines {k}-{len(cell5_lines)} (5. APPLY DYNAMIC RANGE, 6. CONVERT dB TO BRIGHTNESS) -> Team 7")
109
110 OLD_LINE = 'DATA_FILE = "carotid_1_rf.npy"'
111 PARAMS = ["fs", "f0", "c", "pitch", "depth_start_mm"]
112 HANDOFF = {}
113
114 for n in (1, 2):
115     name = f"carotid_{n}"
116     banner(f"Part 1, {name}: run notebook cells 1-4 and cell 5 (first half)")
117     cells = [bm_cells[0], bm_cells[1], bm_cells[2], bm_cells[3], cell5_part1]
118     if n == 2: # change only the file name line, in memory (the notebook file is not edited)
119         assert bm_cells[0].count(OLD_LINE) == 1
120         cells[0] = bm_cells[0].replace(OLD_LINE, 'DATA_FILE = "carotid_2_rf.npy"')
121         print(f" (in memory only) replaced the line {OLD_LINE} by DATA_FILE = \"carotid_2_rf.npy\"")
122     labels = ["cell 1", "cell 2", "cell 3", "cell 4", "cell 5 (first half)"]
123
124     fig_no = [0]
125
126     def show_and_save(*args, _name=name, **kwargs):
127         """Replacement for plt.show(): saves the notebook's current figure to output/."""
128         fig_no[0] += 1
129         save(plt.gcf(), f"{_name}_01_notebook_rf_and_envelope.png",
130              f"{_name}: figure drawn by the notebook itself (code cell 4): RF signal and its Hilbert envelope of the "
131              f"middle scan line.")
132
133     ns = {"__name__": "__main__"}
134     real_show, old_cwd = plt.show, os.getcwd()
135     plt.show = show_and_save
136     os.chdir(INPUT) # the notebook loads the bare file name "carotid_N_rf.npy"
137     try:
138         t0 = time.time()
139         for lab, src in zip(labels, cells):
140             print(f"--- running B_mode.ipynb {lab} ({len(src.strip().splitlines())} lines) ---")
141             exec(compile(src, f"B_mode.ipynb[{lab}]", "exec"), ns)
142         finally:
143             os.chdir(old_cwd)
144             plt.show = real_show
145         print(f"Notebook code finished in {time.time() - t0:.1f} s")
146
147         rf, env, env_n, env_db = ns["rf"], ns["envelope"], ns["envelope_norm"], ns["envelope_db"]
148         print("\nRunner checks on the notebook's variables:")
149         for v in ("rf", "envelope", "envelope_norm", "envelope_db"):
150             a = ns[v]
151             print(f" {v:14s}: shape {a.shape}, dtype {a.dtype}, min {a.min():.6g}, max {a.max():.6g}")
152         print(f"envelope >= |rf| everywhere (tolerance 1e-3 x max): "
153               f"{bool(np.all(env >= np.abs(rf) - 1e-3 * env.max()))}")
154         print(f"envelope max / min = {env.max() / env.min():.0f} -> {20 * np.log10(env.max() / env.min()):.1f} dB "
155               f"between the strongest and the weakest echo")

```

```

156     print(f"   envelope_db: min {env_db.min():.2f} dB, max {env_db.max():.2f} dB (0 dB = peak), "
157           f"median {np.median(env_db):.2f} dB")
158     for lim in (40, 50, 60):
159         print(f"   pixels below -{lim} dB: {100 * np.mean(env_db < -lim):5.2f} %")
160     print(" (envelope_db is NOT clipped here: choosing the dynamic range and clipping is Team 7's
↳ step)")
161
162     # ---- handoff file for Team 7
163     out = OUTPUT / f"reconstructed_{name}.npz"
164     np.savez(out, rf=rf, envelope=env, envelope_norm=env_n, envelope_db=env_db,
165             **{p: np.float64(ns[p]) for p in PARAMS})
166     HANDOFF[name] = out
167     SAVED.append((out.name, f"HANDOFF for Team 7: rf, envelope, envelope_norm, envelope_db + fs, f
↳ 0, c, pitch, "
168                  f"depth_start_mm of {name}"))
169     print(f"\nHandoff for Team 7 (B-mode Image Formation): output/{out.name}")
170     with np.load(out) as d:
171         for key in d.files:
172             a = d[key]
173             extra = f"value {a.item():g}" if a.ndim == 0 else f"min {a.min():.6g}, max {a.max():.6
↳ g)"
174             print(f"   {key:15s} shape {str(a.shape):12s} dtype {str(a.dtype):8s} {extra}")
175
176     # ---- runner figures
177     z = ns["depth_axis_mm"]                                     # notebook cell 4 variabl
↳ e
178     x = (np.arange(rf.shape[1]) - rf.shape[1] / 2) * ns["pitch"] * 1000 # same formula as notebo
↳ ok cell 6
179     ext = [x[0], x[-1], z[-1], z[0]]
180     rf_lim = np.percentile(np.abs(rf), 99)
181
182     fig, ax = plt.subplots(1, 3, figsize=(15, 6.2))
183     im = ax[0].imshow(rf, cmap="gray", extent=ext, vmin=-rf_lim, vmax=rf_lim)
184     ax[0].set_title(f"Input: beamformed RF\n(colour limits +/- 99th percentile of |rf|)", fontsize
↳ =10)
185     plt.colorbar(im, ax=ax[0], label="amplitude")
186     im = ax[1].imshow(env, cmap="gray", extent=ext)
187     ax[1].set_title("Envelope |hilbert(rf)|, linear scale\n(only the strongest echoes are visible)
↳ ", fontsize=10)
188     plt.colorbar(im, ax=ax[1], label="amplitude")
189     im = ax[2].imshow(env_db, cmap="gray", extent=ext, vmin=-60, vmax=0)
190     ax[2].set_title("Log-compressed envelope_db\n(display window -60..0 dB)", fontsize=10)
191     plt.colorbar(im, ax=ax[2], label="dB (0 dB = peak)")
192     for a in ax:
193         a.set_xlabel("lateral (mm)"); a.set_ylabel("depth (mm)")
194     fig.suptitle(f"Runner figure, {name}: RF input -> envelope -> log compression (Team 6 output)"
↳ , fontsize=11)
195     fig.tight_layout()
196     save(fig, f"{name}_02_runner_rf_envelope_db.png",
197           f"Runner figure, {name}: the RF input, the linear envelope and the log-compressed envelop
↳ e (dB).")
198
199     fig, ax = plt.subplots(1, 2, figsize=(13, 5.6), gridspec_kw=dict(width_ratios=[1, 1.25]))
200     im = ax[0].imshow(env_db, cmap="viridis", extent=ext, vmin=env_db.min(), vmax=0)
201     ax[0].set_title(f"envelope_db, full unclipped range\n({env_db.min():.1f} to 0 dB)", fontsize=1
↳ 0)
202     ax[0].set_xlabel("lateral (mm)"); ax[0].set_ylabel("depth (mm)")
203     plt.colorbar(im, ax=ax[0], label="dB")
204     ax[1].hist(env_db.ravel(), bins=120, color="tab:blue", alpha=0.85)
205     for lim, col in ((40, "tab:red"), (50, "tab:orange"), (60, "tab:green")):
206         ax[1].axvline(-lim, color=col, ls="--", lw=1.2, label=f"-{lim} dB ({100 * np.mean(env_db <
↳ -lim):.1f} % below)")
207     ax[1].set_xlabel("envelope_db (dB)"); ax[1].set_ylabel("number of pixels")
208     ax[1].set_title("Histogram of envelope_db values\n(dashed lines: possible dynamic-range limits
↳ Team 7 may choose)",
209                    fontsize=10)
210     ax[1].legend(fontsize=8); ax[1].grid(alpha=0.3)
211     fig.suptitle(f"Runner figure, {name}: the log-compressed data handed to Team 7 (not clipped)",
↳ fontsize=11)
212     fig.tight_layout()
213     save(fig, f"{name}_03_runner_db_full_range_and_histogram.png",
214           f"Runner figure, {name}: envelope_db over its whole range and the histogram of its values
↳ .")
215
216 # =====
217 # PART 2: code/image_reconstruction.ipynb (simulated RF, all code cells, unchanged)
218 # =====

```

```

219 banner("PART 2: simulated RF (code/image_reconstruction.ipynb)")
220 cells = code_cells(CODE / "image_reconstruction.ipynb")
221 print(f"Notebook has {len(cells)} code cells. Running them in order...\n")
222
223
224 def show_and_save_sim(*args, **kwargs):
225     save(plt.gcf(), "sim_A1_notebook_rf_envelope_and_scan_conversion.png",
226         "Figure made by image_reconstruction.ipynb itself: left = band-pass filtered RF of scan 1
↳ line 32 with its "
227         "Hilbert envelope; right = scan-converted (reconstructed) envelope image.")
228
229
230 ns = {"__name__": "__main__"}
231 real_show = plt.show
232 plt.show = show_and_save_sim
233 t0 = time.time()
234 try:
235     for i, src in enumerate(cells):
236         print(f"--- running code cell {i} ({len(src.splitlines())} lines) ---")
237         exec(compile(src, f"image_reconstruction.ipynb[cell {i}]", "exec"), ns)
238 finally:
239     plt.show = real_show
240 print(f"\nNotebook code finished in {time.time() - t0:.1f} s")
241
242 print("\nRunner checks on the notebook's variables:")
243 print(f" simulated raw RF matrix : {ns['raw_rf_input'].shape[0]} samples x {ns['raw_rf_input'].sh
↳ ape[1]} scan lines "
244     f"(fs = {ns['fs']} / 1e6:.0f} MHz, fc = {ns['fc']} / 1e6:.0f} MHz)")
245 print(f" band-pass filter : Butterworth order 2, {ns['low_cut']} / 1e6:.0f}-{ns['high_cut']
↳ / 1e6:.0f} MHz, "
246     f"zero-phase (filtfilt)")
247 print(f" mean of raw RF (DC) : {ns['raw_rf_input'].mean():.4f} -> after DC removal + filter
↳ : "
248     f"{ns['preprocessed_rf'].mean():.2e}")
249 for depth, line, strength in ns["targets"]:
250     e = ns["envelope"][line]
251     kk = np.argmax(e)
252     print(f" target at {depth * 1e3:4.0f} mm on line {line:2d}: envelope peak found at "
253         f"{ns['depth_mm'][kk]:6.2f} mm, peak value {e[kk]:.3f} (reflectivity {strength})")
254 rec = ns["reconstructed_envelope"]
255 print(f" reconstructed image : {rec.shape}, min {rec.min():.3f}, max {rec.max():.3f}, "
256     f"zero-filled (outside sector) pixels {100 * np.mean(rec == 0):.1f} %")
257
258 d, raw, pre, env = ns["depth_mm"], ns["raw_rf_input"], ns["preprocessed_rf"], ns["envelope"]
259 fig, ax = plt.subplots(1, 2, figsize=(13, 4.8))
260 ax[0].plot(d, raw[:, 32], lw=0.7, color="0.4", label="raw simulated RF (with DC bias + noise)")
261 ax[0].plot(d, pre[:, 32], lw=0.9, color="tab:blue", label="after DC removal + 2-8 MHz band-pass")
262 ax[0].set_xlim(10, 20); ax[0].set_xlabel("depth (mm)"); ax[0].set_ylabel("amplitude"); ax[0].grid(
↳ alpha=0.3)
263 ax[0].set_title("Scan line 32, zoom 10-20 mm (target at 15 mm)"); ax[0].legend(fontsize=8)
264 nb10 = raw.shape[0] // 10
265 blk = np.abs(raw[:nb10 * 10] - raw.mean()).reshape(nb10, 10, -1).max(axis=-1) # peak |RF| per 10 s
↳ amples
266 im = ax[1].imshow(blk, aspect="auto", cmap="gray", vmin=0, vmax=0.8, extent=[0, raw.shape[1] - 1,
↳ d[-1], d[0]])
267 ax[1].set_xlabel("scan line index"); ax[1].set_ylabel("depth (mm)")
268 ax[1].set_title("Simulated raw RF matrix:\npeak |RF| per 10 samples, DC removed for display", font
↳ size=10)
269 plt.colorbar(im, ax=ax[1], label="|amplitude|")
270 fig.suptitle("Runner figure (Part 2): the simulated raw RF input of image_reconstruction.ipynb", fonts
↳ size=11, y=1.04)
271 save(fig, "sim_A2_runner_raw_rf_input.png",
272     "Runner figure (Part 2): the simulated raw RF for line 32 before/after filtering, and the ful
↳ l RF matrix.")
273
274 env_db = 20 * np.log10(env / env.max() + 1e-6)
275 rec_db = 20 * np.log10(np.maximum(rec, 1e-6) / rec.max())
276 fig, ax = plt.subplots(1, 3, figsize=(16, 5.6), gridspec_kw=dict(width_ratios=[1, 1, 1.35]))
277 ax[0].imshow(env, aspect="auto", cmap="gray", extent=[0, 63, d[-1], d[0]])
278 ax[0].set_title("Envelope (linear), lines x depth"); ax[0].set_xlabel("scan line"); ax[0].set_ylabel
↳ el("depth (mm)")
279 ax[1].imshow(env_db, aspect="auto", cmap="gray", vmin=-40, vmax=0, extent=[0, 63, d[-1], d[0]])
280 ax[1].set_title("Envelope in dB (display window -40..0 dB)"); ax[1].set_xlabel("scan line")
281 xc, zc = ns["x_cart"], ns["z_cart"]
282 im = ax[2].imshow(rec_db, cmap="gray", vmin=-40, vmax=0, extent=[xc[0], xc[-1], zc[-1], zc[0]], as
↳ pect="equal")
283 ax[2].set_title("Scan-converted image in dB\n(display window -40..0 dB)"); ax[2].set_xlabel("later

```

```

    al (mm)")
284 ax[2].set_ylabel("depth (mm)"); plt.colorbar(im, ax=ax[2], label="dB")
285 fig.suptitle("Runner figure (Part 2): envelope before and after scan conversion, linear and in dB
    "
286             "(the notebook itself stops at the linear envelope)", fontsize=10)
287 save(fig, "sim_A3_runner_envelope_db_and_scan_converted.png",
288       "Runner figure (Part 2): envelope as lines x depth (linear and in dB) and the scan-converted
    sector image in dB.")
289
290 # =====
291 banner("SUMMARY")
292 for name, path in HANDOFF.items():
293     with np.load(path) as dd:
294         print(f"{name}: envelope_db {dd['envelope_db'].shape} {dd['envelope_db'].dtype}, "
295               f"{dd['envelope_db'].min():.2f} to {dd['envelope_db'].max():.2f} dB")
296     print(f"  Handoff for Team 7 (B-mode Image Formation): output/{path.name}")
297 print(f"Part 2: reconstructed (scan-converted) envelope {ns['reconstructed_envelope'].shape}; enve
    lope peaks found for "
298       f"all {len(ns['targets'])} targets.")
299 print(f"Files written to output/: {len(SAVED) + 2} (including console_log.txt and console_screensh
    ot.png)")
300 print(f"Total run time: {time.time() - t_start:.1f} s")
301
302 # ----- console log + screenshot
303 sys.stdout = REAL_STDOUT
304 text = LOG.getvalue()
305 (OUTPUT / "console_log.txt").write_text(text, encoding="utf-8")
306
307
308 def console_png(text, out_png, width=112):
309     lines = []
310     for ln in text.rstrip("\n").splitlines():
311         while len(ln) > width:
312             lines.append(ln[:width]); ln = "  " + ln[width:]
313         lines.append(ln)
314     h = 0.6 + 0.16 * len(lines)
315     fig = plt.figure(figsize=(0.083 * width + 0.4, h))
316     fig.patch.set_facecolor("#1e1e1e")
317     fig.text(0.01, 1 - 0.28 / h, "Terminal: python3 run_team6.py", color="#9a9a9a", fontsize=9, fa
    mily="monospace",
318            va="center")
319     for i, ln in enumerate(lines):
320         fig.text(0.01, 1 - (0.55 + 0.16 * i) / h, ln, color="#e6e6e6", fontsize=8.4, family="monos
    pace", va="top")
321     fig.savefig(out_png, dpi=110, facecolor=fig.get_facecolor())
322     plt.close(fig)
323
324
325 console_png(text, OUTPUT / "console_screenshot.png")
326 print("Saved output/console_log.txt and output/console_screenshot.png")

```

4 Input

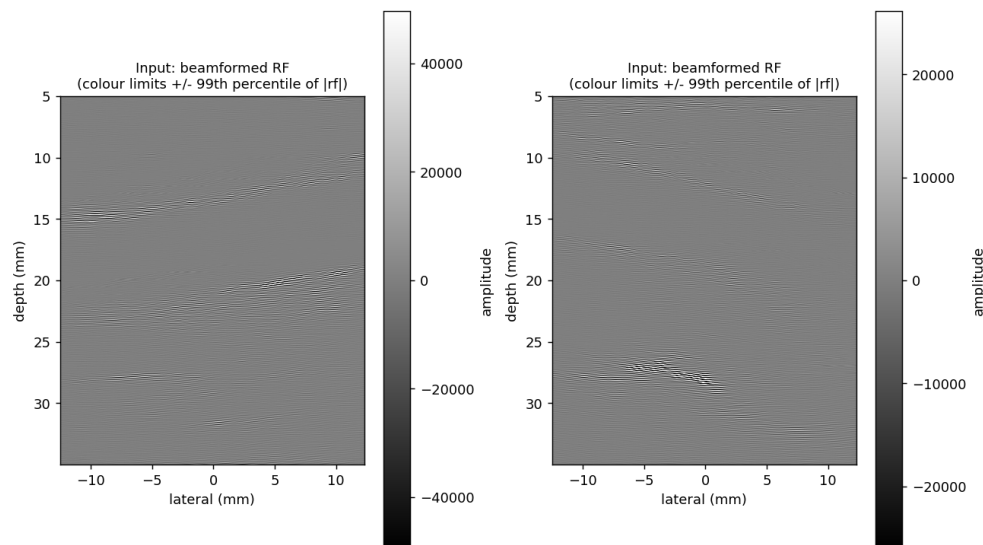
What goes into this stage

4.1 Real in-vivo carotid RF (Part 1)

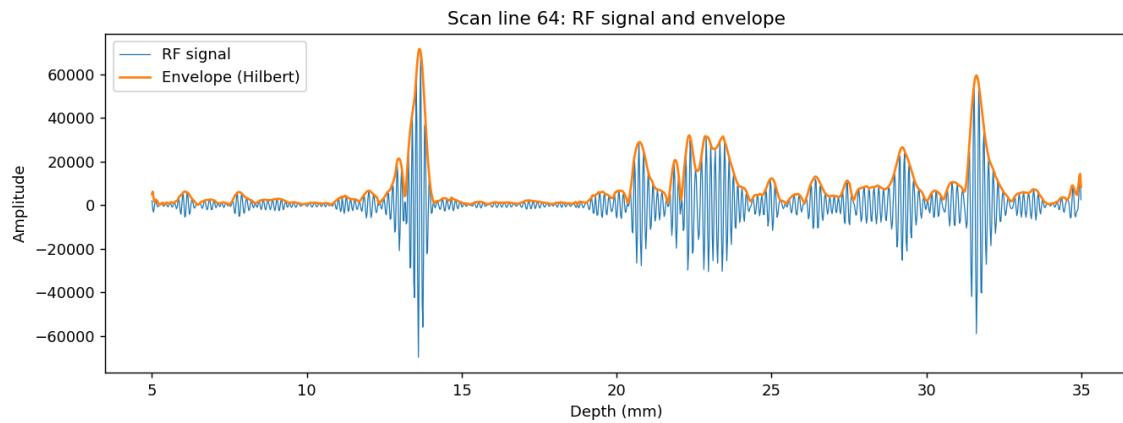
The input is beamformed RF data of a human carotid artery (the large blood vessel in the neck), longitudinal view. According to the notebook's description:

- Data source: EPFL LTS5 public repository **us-non-stationary-deconv** (github.com/LTS5/us-non-stationary-deconv), A. Besson et al., 'A Physical Model of Non-stationary Blur in Ultrasound Imaging', IEEE Trans. Computational Imaging.
- Probe: **128-element linear array L12-50**, centre frequency **5 MHz**, single plane-wave transmission, sampling frequency **fs = 31.25 MHz**.
- The raw channel data was band-pass filtered (2-8 MHz) and beamformed with delay-and-sum to obtain the RF data.

File	Shape (samples x lines)	dtype	Min	Max	Depth
input/carotid_1_rf.npy	1218 x 128	float32	-162577.23	166418.19	5.0-35.0 mm
input/carotid_2_rf.npy	1218 x 128	float32	-130417.8	139392.75	5.0-35.0 mm



The two input RF frames (left: carotid_1, right: carotid_2), cut from the runner figures carotid_N_02. Gray = RF amplitude (colour limits +/- 99th percentile of |rf|). The fine stripes are the 5 MHz oscillation; the bright tilted bands are the vessel walls.

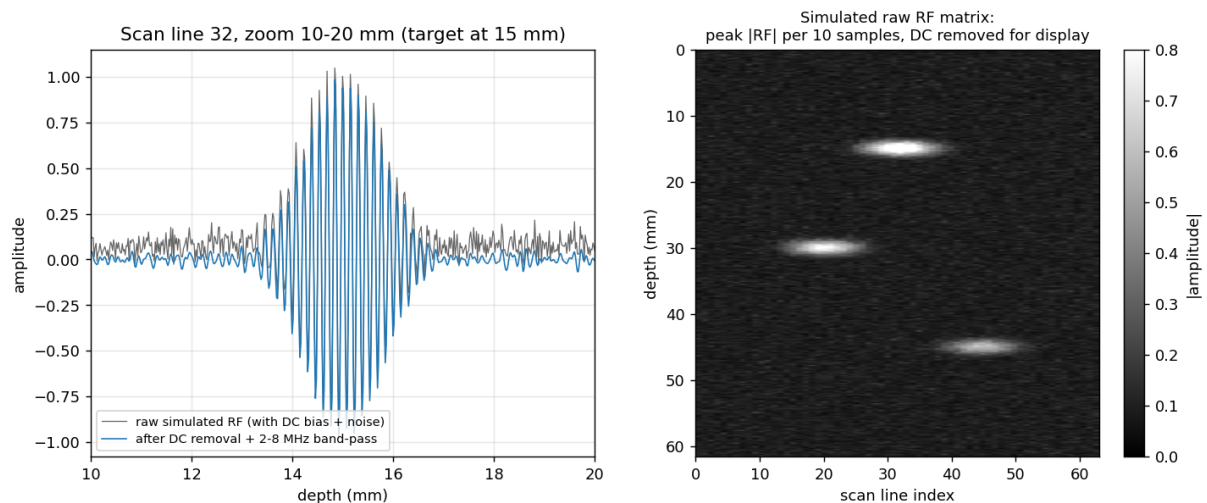


One input scan line (line 64 of carotid_1, blue) drawn by notebook cell 4, with its envelope (orange).

4.2 Simulated RF (Part 2)

image_reconstruction.ipynb needs no input file: it creates a 3201 x 64 RF matrix ($f_s = 40$ MHz, $f_c = 5$ MHz) with three point targets at 15, 30 and 45 mm, a DC offset of 0.08 and Gaussian noise.

Runner figure (Part 2): the simulated RF input of image_reconstruction.ipynb

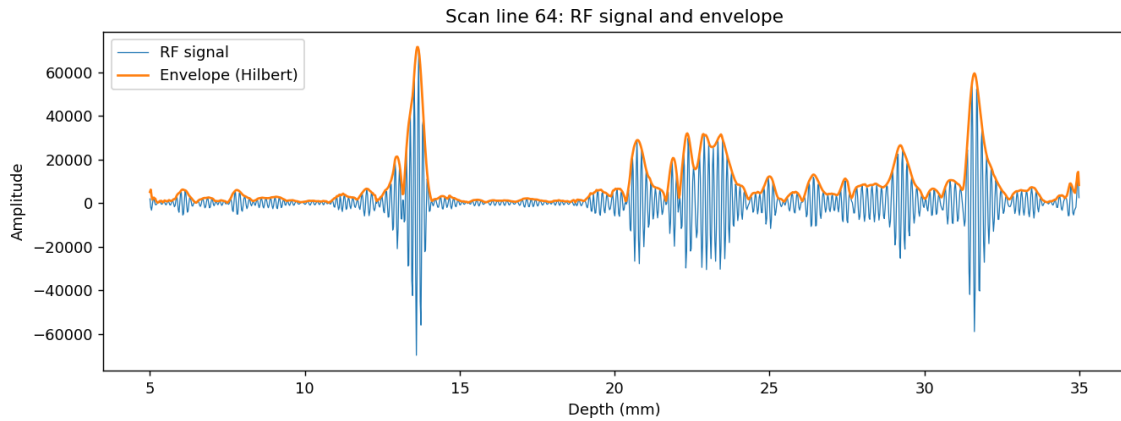


sim_A2 (runner figure): left, raw RF of scan line 32 near the 15 mm target (grey, with DC offset and noise) and the same line after DC removal and 2-8 MHz band-pass (blue). Right, the whole simulated RF matrix (peak $|RF|$ per 10 samples).

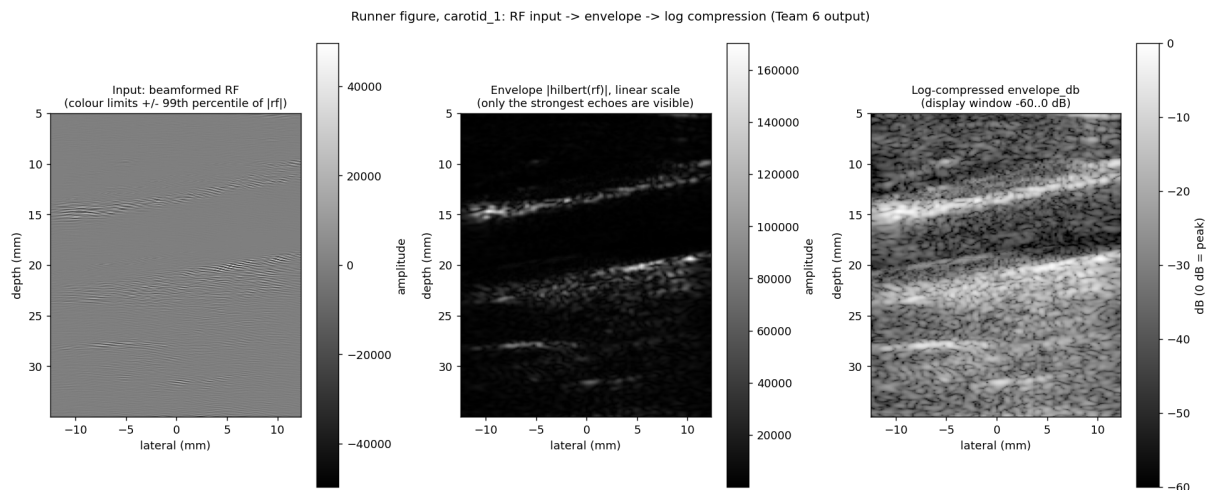
5 Output

Everything below was produced by run_team6.py in this run

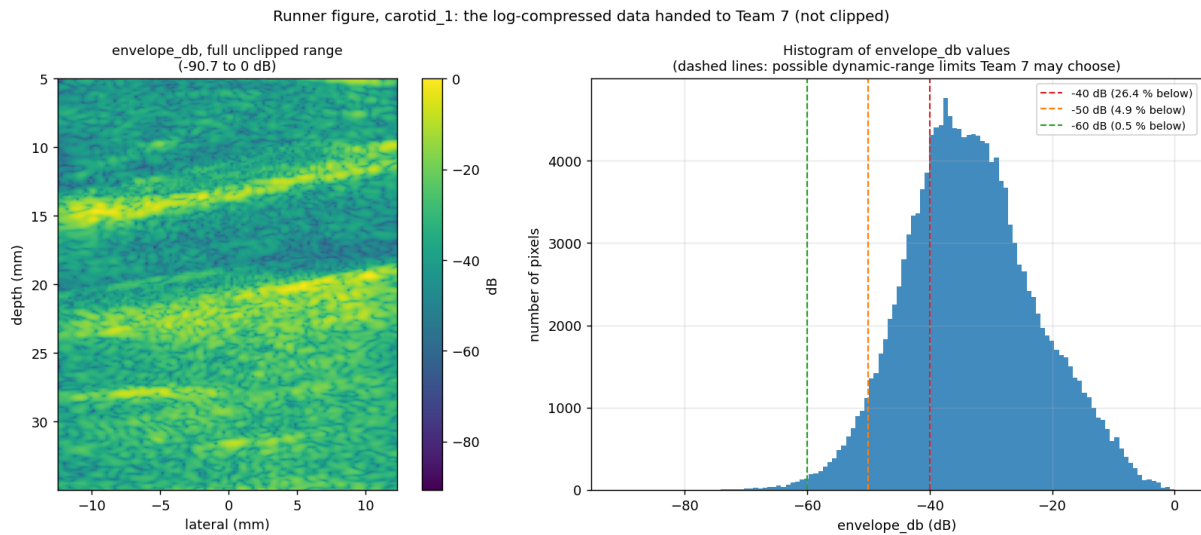
5.1 Carotid figures (Part 1)



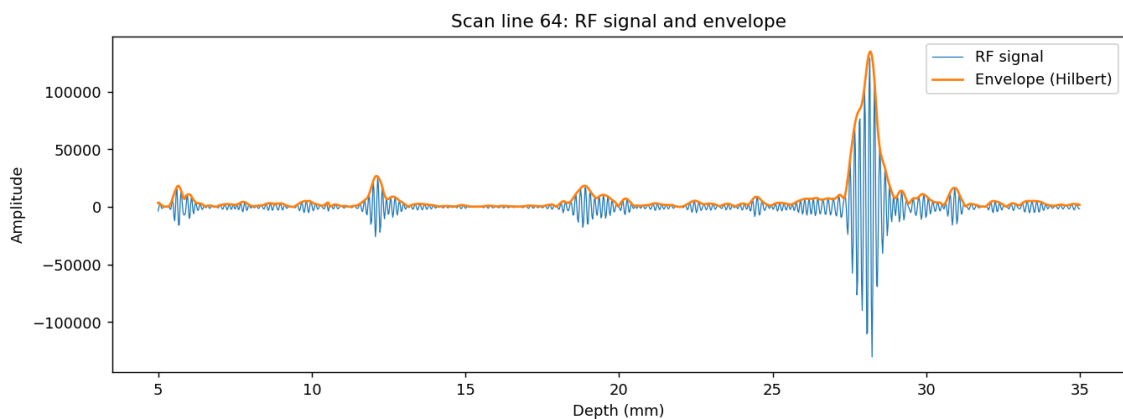
carotid_1_01 (notebook cell 4 figure): scan line 64, RF signal (blue) and its Hilbert envelope (orange), which follows the peaks of the wave.



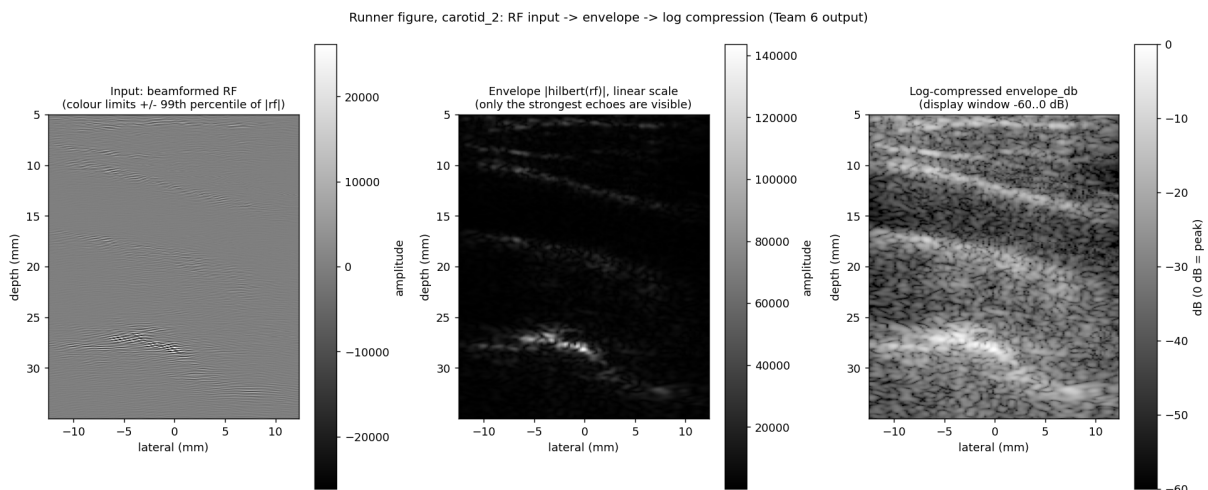
carotid_1_02 (runner figure): RF input; envelope on a linear scale (only the strongest wall echoes are visible); log-compressed envelope_db shown in a -60..0 dB display window, where the vessel walls, the dark lumen and the tissue speckle are all visible. Only 0.50 % of the pixels lie below -60 dB.



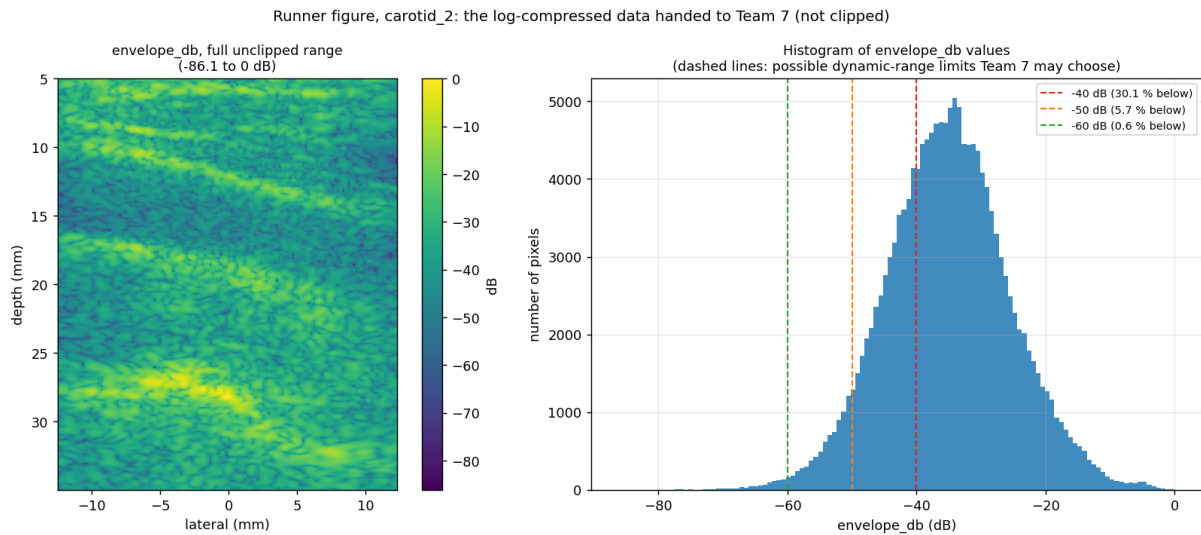
carotid_1_03 (runner figure): left, envelope_db over its full unclipped range (-90.65 to 0 dB); right, histogram of the dB values with the median at -33.75 dB. The dashed lines show where Team 7 could cut: 26.44 % of pixels are below -40 dB, 4.90 % below -50 dB, 0.50 % below -60 dB.



carotid_2_01 (notebook cell 4 figure): scan line 64, RF signal (blue) and its Hilbert envelope (orange), which follows the peaks of the wave.



carotid_2_02 (runner figure): RF input; envelope on a linear scale (only the strongest wall echoes are visible); log-compressed envelope_db shown in a -60..0 dB display window, where the vessel walls, the dark lumen and the tissue speckle are all visible. Only 0.63 % of the pixels lie below -60 dB.



carotid_2_03 (runner figure): left, envelope_db over its full unclipped range (-86.10 to 0 dB); right, histogram of the dB values with the median at -35.14 dB. The dashed lines show where Team 7 could cut: 30.11 % of pixels are below -40 dB, 5.74 % below -50 dB, 0.63 % below -60 dB.

Frame	Envelope min	Envelope max	max/min	envelope_db range	Median	< -40 dB	< -50 dB	< -60 dB
carotid_1	5.00	170356	34100	-90.65 to 0 dB	-33.75 dB	26.44 %	4.90 %	0.50 %
carotid_2	7.11	143632	20189	-86.10 to 0 dB	-35.14 dB	30.11 %	5.74 %	0.63 %

5.2 Handoff files for Team 7

For each frame the runner saves the notebook's own variables, exactly as computed (no casting), with `np.savez` (not compressed). `envelope_db` is the log-compressed envelope in dB, 0 dB = peak, **not yet clipped**. The cell-2 parameters are stored as numpy float64 scalars.

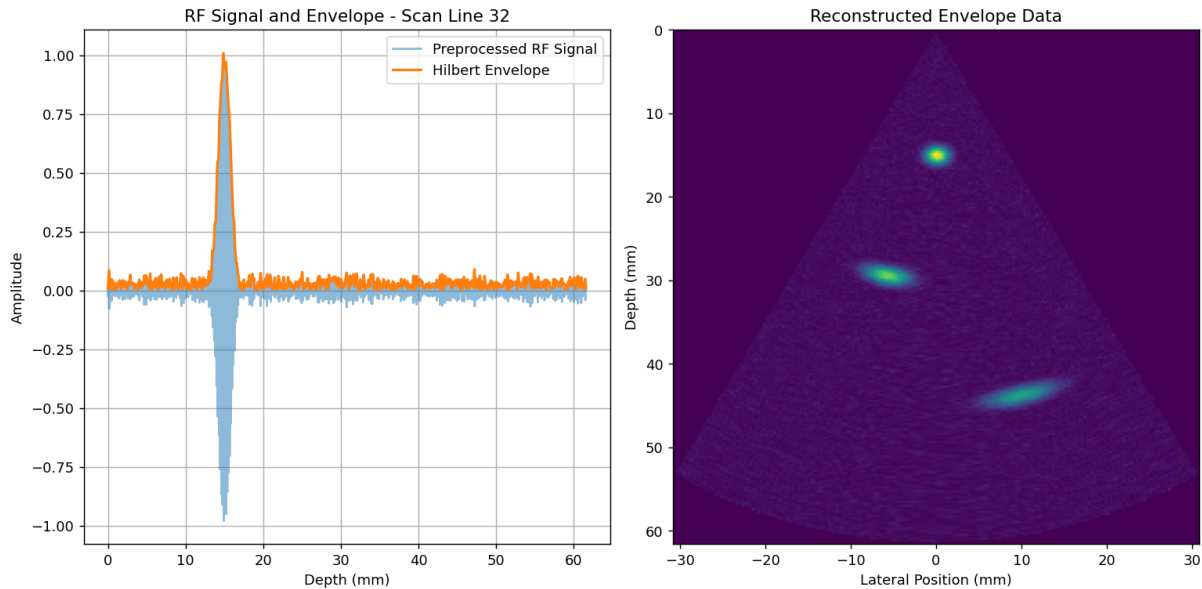
Key	Shape	dtype	carotid_1	carotid_2
rf	(1218, 128)	float32	-162577 to 166418	-130418 to 139393
envelope	(1218, 128)	float32	4.99583 to 170356	7.11421 to 143632
envelope_norm	(1218, 128)	float32	2.93259e-05 to 1	4.95308e-05 to 1
envelope_db	(1218, 128)	float32	-90.655 to 0	-86.1025 to 0
fs	()	float64	3.125e+07	3.125e+07
f0	()	float64	5e+06	5e+06
c	()	float64	1540	1540
pitch	()	float64	0.0001953	0.0001953
depth_start_mm	()	float64	5	5

Files: `output/reconstructed_carotid_1.npz` and `output/reconstructed_carotid_2.npz`.
Reading them in Team 7's code:

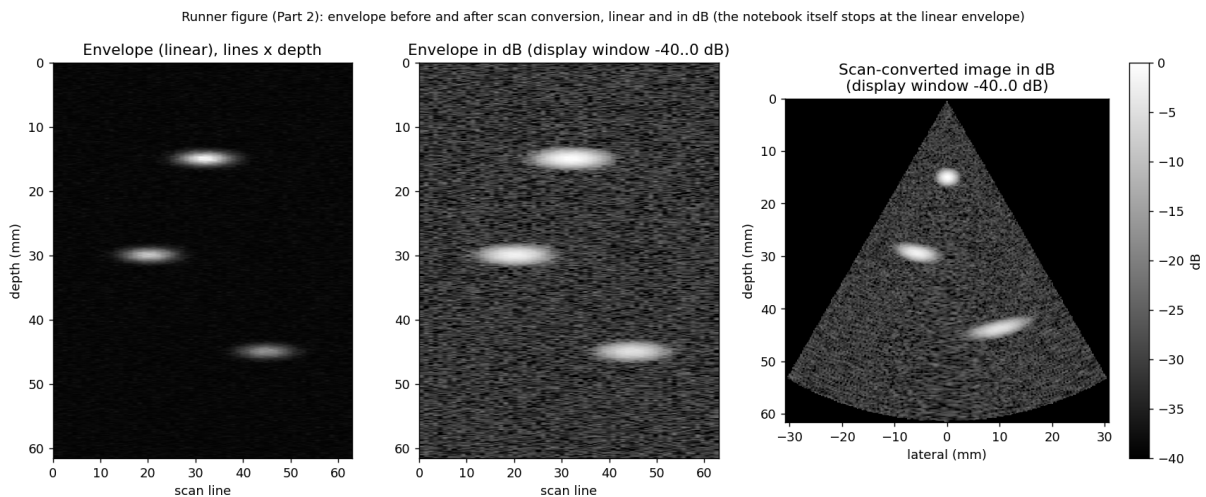
example

```
1 d = np.load("reconstructed_carotid_1.npz")
2 envelope_db = d["envelope_db"] # (1218, 128) float32, dB, 0 = peak
3 fs, c = float(d["fs"]), float(d["c"])
```

5.3 Simulated-RF figures (Part 2)



sim_A1 (the notebook's own figure): left, band-pass filtered RF of scan line 32 with its Hilbert envelope; the envelope peaks at 14.88 mm (target at 15 mm). Right, the scan-converted envelope on a 200 x 250 grid: three bright spots in a fan from -30 to +30 degrees.



sim_A3 (runner figure): envelope as lines x depth on a linear scale, the same in dB (display window -40..0 dB), and the scan-converted image in dB. The notebook itself stops at the linear envelope.

Measured: the envelope peaks are at **14.88 mm**, **29.91 mm** and **45.05 mm** for targets at 15, 30 and 45 mm, so the time-to-depth conversion is right to about 0.1 mm. The peak values 1.008, 0.825 and 0.581 follow the reflectivities 1.0, 0.8 and 0.6.

5.4 Console output

output/console_log.txt (real captured text)

```
=====
TEAM 6: IMAGE RECONSTRUCTION (NITK-UsoundSim)
=====
```

```

Python 3.12.4, numpy 1.26.4, matplotlib 3.8.3
Stage order: Image Reconstruction (Team 6: Hilbert envelope + log compression)
              -> B-mode Image Formation (Team 7: dynamic range + grayscale image)
              -> Post-Image Processing (Team 8: despeckling)

=====
PART 1: real in-vivo carotid RF (code/B_mode.ipynb, steps 1-4)
=====
B_mode.ipynb has 9 code cells. This stage runs cells 1, 2, 3, 4 and the first half of cell 5.
Code cell 5 split at line 15 (the '=' line above '# 5. APPLY DYNAMIC RANGE'):
  first half = lines 1-14 (3. NORMALIZE ENVELOPE, 4. LOG COMPRESSION) -> run here
  second half = lines 15-29 (5. APPLY DYNAMIC RANGE, 6. CONVERT dB TO BRIGHTNESS) -> Team 7

=====
Part 1, carotid_1: run notebook cells 1-4 and cell 5 (first half)
=====
--- running B_mode.ipynb cell 1 (18 lines) ---
RF data shape: (1218, 128)
Depth samples: 1218
Scan lines : 128
Minimum: -162577.23
Maximum: 166418.19
--- running B_mode.ipynb cell 2 (11 lines) ---
--- running B_mode.ipynb cell 3 (12 lines) ---
Envelope shape: (1218, 128)
Minimum: 4.995831
Maximum: 170355.81
--- running B_mode.ipynb cell 4 (15 lines) ---
  saved output/carotid_1_01_notebook_rf_and_envelope.png
--- running B_mode.ipynb cell 5 (first half) (12 lines) ---
Notebook code finished in 0.8 s

Runner checks on the notebook's variables:
rf           : shape (1218, 128), dtype float32, min -162577, max 166418
envelope     : shape (1218, 128), dtype float32, min 4.99583, max 170356
envelope_norm : shape (1218, 128), dtype float32, min 2.93259e-05, max 1
envelope_db  : shape (1218, 128), dtype float32, min -90.655, max 0
envelope >= |rf| everywhere (tolerance 1e-3 x max): True
envelope max / min = 34100 -> 90.7 dB between the strongest and the weakest echo
envelope_db: min -90.65 dB, max 0.00 dB (0 dB = peak), median -33.75 dB
pixels below -40 dB: 26.44 %
pixels below -50 dB: 4.90 %
pixels below -60 dB: 0.50 %
(envelope_db is NOT clipped here: choosing the dynamic range and clipping is Team 7's step)

Handoff for Team 7 (B-mode Image Formation): output/reconstructed_carotid_1.npz
rf           shape (1218, 128) dtype float32 min -162577, max 166418
envelope     shape (1218, 128) dtype float32 min 4.99583, max 170356
envelope_norm shape (1218, 128) dtype float32 min 2.93259e-05, max 1
envelope_db  shape (1218, 128) dtype float32 min -90.655, max 0
fs           shape () dtype float64 value 3.125e+07
f0           shape () dtype float64 value 5e+06
c            shape () dtype float64 value 1540
pitch        shape () dtype float64 value 0.0001953
depth_start_mm shape () dtype float64 value 5
saved output/carotid_1_02_runner_rf_envelope_db.png
saved output/carotid_1_03_runner_db_full_range_and_histogram.png

=====
Part 1, carotid_2: run notebook cells 1-4 and cell 5 (first half)
=====
  (in memory only) replaced the line DATA_FILE = "carotid_1_rf.npy" by DATA_FILE = "carotid_2_rf.npy"
--- running B_mode.ipynb cell 1 (18 lines) ---
RF data shape: (1218, 128)
Depth samples: 1218
Scan lines : 128
Minimum: -130417.8
Maximum: 139392.75
--- running B_mode.ipynb cell 2 (11 lines) ---
--- running B_mode.ipynb cell 3 (12 lines) ---
Envelope shape: (1218, 128)
Minimum: 7.1142106
Maximum: 143632.12
--- running B_mode.ipynb cell 4 (15 lines) ---

```

```

    saved output/carotid_2_01_notebook_rf_and_envelope.png
--- running B_mode.ipynb cell 5 (first half) (12 lines) ---
Notebook code finished in 0.1 s

Runner checks on the notebook's variables:
rf           : shape (1218, 128), dtype float32, min -130418, max 139393
envelope     : shape (1218, 128), dtype float32, min 7.11421, max 143632
envelope_norm : shape (1218, 128), dtype float32, min 4.95308e-05, max 1
envelope_db  : shape (1218, 128), dtype float32, min -86.1025, max 0
envelope >= |rf| everywhere (tolerance 1e-3 x max): True
envelope max / min = 20189 -> 86.1 dB between the strongest and the weakest echo
envelope_db: min -86.10 dB, max 0.00 dB (0 dB = peak), median -35.14 dB
pixels below -40 dB: 30.11 %
pixels below -50 dB: 5.74 %
pixels below -60 dB: 0.63 %
(envelope_db is NOT clipped here: choosing the dynamic range and clipping is Team 7's step)

Handoff for Team 7 (B-mode Image Formation): output/reconstructed_carotid_2.npz
rf           shape (1218, 128) dtype float32 min -130418, max 139393
envelope     shape (1218, 128) dtype float32 min 7.11421, max 143632
envelope_norm shape (1218, 128) dtype float32 min 4.95308e-05, max 1
envelope_db  shape (1218, 128) dtype float32 min -86.1025, max 0
fs           shape () dtype float64 value 3.125e+07
f0           shape () dtype float64 value 5e+06
c            shape () dtype float64 value 1540
pitch        shape () dtype float64 value 0.0001953
depth_start_mm shape () dtype float64 value 5
saved output/carotid_2_02_runner_rf_envelope_db.png
saved output/carotid_2_03_runner_db_full_range_and_histogram.png

=====
PART 2: simulated RF (code/image_reconstruction.ipynb)
=====
Notebook has 3 code cells. Running them in order...

--- running code cell 0 (1 lines) ---
--- running code cell 1 (350 lines) ---
    saved output/sim_A1_notebook_rf_envelope_and_scan_conversion.png
Reconstructed envelope shape: (250, 200)
Depth range: 0.0 to 61.599999999999994 mm
Lateral range: -30.799999999999994 to 30.799999999999994 mm
--- running code cell 2 (0 lines) ---

Notebook code finished in 2.4 s

Runner checks on the notebook's variables:
simulated raw RF matrix : 3201 samples x 64 scan lines (fs = 40 MHz, fc = 5 MHz)
band-pass filter         : Butterworth order 2, 2-8 MHz, zero-phase (filtfilt)
mean of raw RF (DC)      : 0.0800 -> after DC removal + filter: 7.45e-06
target at 15 mm on line 32: envelope peak found at 14.88 mm, peak value 1.008 (reflectivity 1.0)
target at 30 mm on line 20: envelope peak found at 29.91 mm, peak value 0.825 (reflectivity 0.8)
target at 45 mm on line 45: envelope peak found at 45.05 mm, peak value 0.581 (reflectivity 0.6)
reconstructed image      : (250, 200), min 0.000, max 0.992, zero-filled (outside sector) pixels 48.1 %
saved output/sim_A2_runner_raw_rf_input.png
saved output/sim_A3_runner_envelope_db_and_scan_converted.png

=====
SUMMARY
=====
carotid_1: envelope_db (1218, 128) float32, -90.65 to 0.00 dB
    Handoff for Team 7 (B-mode Image Formation): output/reconstructed_carotid_1.npz
carotid_2: envelope_db (1218, 128) float32, -86.10 to 0.00 dB
    Handoff for Team 7 (B-mode Image Formation): output/reconstructed_carotid_2.npz
Part 2: reconstructed (scan-converted) envelope (250, 200); envelope peaks found for all 3 targets.
Files written to output/: 13 (including console_log.txt and console_screenshot.png)
Total run time: 5.5 s

```

The same text is saved as a picture in output/console_screenshot.png.

6 How to run

Step by step, on macOS, Linux or Windows

1. Install Python 3.9 or newer from <https://www.python.org> (Windows: tick 'Add Python to PATH').
2. Open a terminal in the folder Team_6_Image_Reconstruction (macOS: right-click the folder in Finder > New Terminal at Folder; Windows: type cmd in the Explorer address bar).
3. Install the packages once: `python3 -m pip install -r requirements.txt` (Windows: `python -m pip install -r requirements.txt`).
4. Run: `python3 run_team6.py` (Windows: `python run_team6.py`).
5. Look in output/: the carotid_N_* and sim_A* figures, the handoff files `reconstructed_carotid_N.npz`, `console_log.txt` and `console_screenshot.png`.

Expected runtime: about 10-15 seconds (this run: 5.5 s reported by the script). Most of it is the griddata interpolation in Part 2. No internet is needed.

Opening the notebooks yourself

- Jupyter: `python3 -m pip install jupyter`, then `jupyter notebook`. B_mode.ipynb loads the bare file name `carotid_1_rf.npy`, and a copy of it is already in code/ next to the notebook, so code/B_mode.ipynb runs directly. Run cells 1-5 up to section 4 for this stage.
- For carotid_2: copy `input/carotid_2_rf.npy` into code/ and change DATA_FILE in cell 1 to "`carotid_2_rf.npy`" (run_team6.py does the same change in memory only).
- Colab: upload the .ipynb at <https://colab.research.google.com>; for B_mode.ipynb also upload the .npy file.

Troubleshooting

Problem	What to do
python3 / python not found	Install Python; on Windows try <code>py run_team6.py</code> .
No module named numpy / scipy / matplotlib	Repeat step 3 (<code>pip install -r requirements.txt</code>).
FileNotFoundError for input/ or code/	Keep the folder structure unchanged (<code>input/carotid_1_rf.npy</code> and <code>carotid_2_rf.npy</code> must exist).
pip says 'externally managed environment'	<code>python3 -m venv venv</code> ; <code>source venv/bin/activate</code> (Windows: <code>venv\Scripts\activate</code>); then step 3.
No plot windows appear	Normal: plots are saved as PNG files in output/.

7 Check yourself

Viva-style questions with short answers

How does the Hilbert transform give the envelope?

It builds the analytic signal $s + jH\{s\}$; its magnitude $\sqrt{s^2 + H\{s\}^2}$ is a smooth curve through the peaks of the RF wave. For $A \cos(\omega t)$ it gives exactly A .

Why normalise the envelope to its peak before taking the log?

So that the strongest echo becomes $1 = 0$ dB and every other value is 'dB below the peak', independent of the receiver's arbitrary units and gain.

Why use $20 \log_{10}$ and not the linear envelope?

Echo strengths span a huge range (carotid_1: 90.7 dB, ratio 34100). On a linear scale only the strongest wall echoes are visible; the log turns equal ratios into equal steps so weak and strong echoes are visible together. 20 (not 10) because these are amplitudes, not powers.

What does -40 dB mean?

The echo amplitude is 100 times smaller than the strongest echo of the frame.

Why is `envelope_db` handed to Team 7 without clipping, and what does Team 7 do with it?

Choosing the dynamic range is an image-formation decision. Team 7 clips to its dynamic range (for example -60..0 dB) and maps the values to brightness 0-1 to form the B-mode image.

Why does `image_reconstruction.ipynb` use `filtfilt` and `griddata`?

`filtfilt` filters forward and backward, so the band-pass causes no depth shift (zero phase). `griddata` interpolates the fan-shaped (r, theta) samples onto a regular x-z pixel grid (scan conversion).